

Week 4:

```
from dotenv import load_dotenv
import os
import anthropic

load_dotenv()

client =
anthropic.Anthropic(api_key=os.getenv("ANTHROPIC_API_KEY"))

def add_user_message(messages, text):
    user_message = {"role": "user", "content": text}
    messages.append(user_message)

def add_assistant_message(messages, text):
    assistant_message = {"role": "assistant", "content": text}
    messages.append(assistant_message)

def chat(messages):
    message = client.messages.create(
        model="claude-haiku-4-5",
        max_tokens=1000,
        messages=messages,
    )
    return message.content[0].text

messages = []

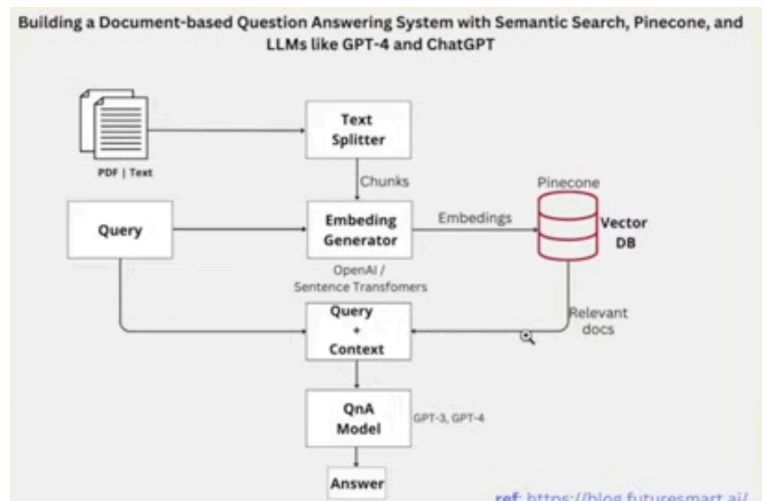
while True:
    user_input = input("> ")
    print(">", user_input)
    add_user_message(messages, user_input)
    answer = chat(messages)
    add_assistant_message(messages, answer)
    print("----")
    print(answer)
    print("----")
```

Vector databases:

They act as an embedding store. Embedding group similar objects in space.

Use cases: recommendation systems, search engines, Gen AI, memory for LLMs

Uses cosine similarity to find the closest sentences and the database then returns this.



```

import chromadb

chroma_client = chromadb.Client()

collections = chroma_client.create_collection(name =
"my_collection")

collections.add(
    documents = ["my name is Anondee", "my name is not Anondee"],
    metadatas = [{"source": "my_source"}, {"source": "my_source"}],
    ids = ["id1", "id2"]
)

results = collections.query(
    query_texts = ["what is my name"],
    n_results = 1
)

print(results)

```

Document QnA

```

import os

#pip install pypdf
#export HNSWLIB_NO_NATIVE = 1

from langchain.document_loaders import PyPDFLoader, TextLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings.openai import OpenAIEmbeddings
from langchain.vectorstores import Chroma
from langchain.chains import RetrievalQAWithSourcesChain
from langchain.chat_models import ChatOpenAI
import chainlit as cl
from chainlit.types import AskFileResponse

text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000,
chunk_overlap=100)
embeddings = OpenAIEmbeddings()

welcome_message = """Welcome to the Chainlit PDF QA demo! To get
started:
1. Upload a PDF or text file
2. Ask a question about the file
"""

```

```

def process_file(file: AskFileResponse):
    import tempfile

    if file.type == "text/plain":
        Loader = TextLoader
    elif file.type == "application/pdf":
        Loader = PyPDFLoader

    with tempfile.NamedTemporaryFile() as tempfile:
        tempfile.write(file.content)
        loader = Loader(tempfile.name)
        documents = loader.load()
        docs = text_splitter.split_documents(documents)
        for i, doc in enumerate(docs):
            doc.metadata["source"] = f"source_{i}"
        return docs

def get_docsearch(file: AskFileResponse):
    docs = process_file(file)

    # Save data in the user session
    cl.user_session.set("docs", docs)

    # Create a unique namespace for the file

    docsearch = Chroma.from_documents(
        docs, embeddings
    )
    return docsearch

@cl.on_chat_start
async def start():
    # Sending an image with the local file path
    await cl.Message(content="You can now chat with your
pdfs.>").send()
    files = None
    while files is None:
        files = await cl.AskFileMessage(
            content=welcome_message,
            accept=["text/plain", "application/pdf"],
            max_size_mb=20,
            timeout=180,
        ).send()

    file = files[0]

    msg = cl.Message(content=f"Processing `{file.name}`...")
    await msg.send()

```

```

# No async implementation in the Pinecone client, fallback to
sync
docsearch = await cl.make_async(get_docsearch)(file)

chain = RetrievalQAWithSourcesChain.from_chain_type(
    ChatOpenAI(temperature=0, streaming=True),
    chain_type="stuff",
    retriever=docsearch.as_retriever(max_tokens_limit=4097),
)

# Let the user know that the system is ready
msg.content = f"`{file.name}` processed. You can now ask
questions!"
await msg.update()

cl.user_session.set("chain", chain)

@cl.on_message
async def main(message):
    chain = cl.user_session.get("chain") # type:
RetrievalQAWithSourcesChain
    cb = cl.AsyncLangchainCallbackHandler(
        stream_final_answer=True, answer_prefix_tokens=["FINAL",
"ANSWER"]
    )
    cb.answer_reached = True
    res = await chain.acall(message, callbacks=[cb])

    answer = res["answer"]
    sources = res["sources"].strip()
    source_elements = []

    # Get the documents from the user session
    docs = cl.user_session.get("docs")
    metadatas = [doc.metadata for doc in docs]
    all_sources = [m["source"] for m in metadatas]

    if sources:
        found_sources = []

        # Add the sources to the message
        for source in sources.split(","):
            source_name = source.strip().replace(".", "")
            # Get the index of the source
            try:
                index = all_sources.index(source_name)
            except ValueError:
                continue
            text = docs[index].page_content
            found_sources.append(source_name)
            # Create the text element referenced in the message

```

```
        source_elements.append(cl.Text(content=text,
name=source_name))

    if found_sources:
        answer += f"\nSources: {' ', ' '.join(found_sources)}"
    else:
        answer += "\nNo sources found"

    if cb.has_streamed_final_answer:
        cb.final_stream.elements = source_elements
        await cb.final_stream.update()
    else:
        await cl.Message(content=answer,
elements=source_elements).send()
```